

for each allocation may be infeasible in practice. Further, since migration takes a significant amount of time, any measurement-based strategy that tries to learn the power models for all used application mixes is also practically infeasible in a dynamic consolidation setting.

We concluded that the power-capping and power-performance tradeoff frameworks, though interesting in theory, are not feasible in practice as they need one to compute the exact power consumed by an application. However, we note that an algorithm to solve the power minimization problem does not need actual power numbers. The problem that the power minimization framework solves is to minimize power, without really knowing what the exact power would be. Hence, an algorithm that can be listed out as a sequence of steps, where each step is a decision problem for some application to determine which of the candidate servers the application should be placed on, does not need estimates of power values. Instead, if the algorithm can figure out which server minimizes the incremental increase in total power due to the new application being placed, then it can place the application appropriately.

One may, however, note that this approach restricts the algorithm choices to only those algorithms that take a local view of the problem, and hence can be locally optimal in the best case. Further, one still needs to solve the ordering problem, where for any given placement of $N - 1$ applications on M servers, we should be able to estimate the best server (in terms of power minimization) to place the N^{th} application. We next investigate two properties such that even if any one of them holds, one can make this decision.

Definition 1. *Ordering Property: For any two applications VM_i and $VM_{i'}$ and servers S_j and $S_{j'}$ at loads ρ_j and $\rho_{j'}$ respectively, if the server S_j is more power efficient than $S_{j'}$ for VM_i , then S_j is more power-efficient than $S_{j'}$ for all $VM_{i'}$ as well. Hence, the slopes for any two servers satisfy a uniform ordering across all applications.*

Definition 2. *Background Independence Property: An application and a server are said to satisfy that Background Independence Property if the incremental power drawn due to the application on the server depends only on the background load intensity and is independent of the traffic mix in the background.*

We next investigate if these properties can be made to hold in certain situations where we know about the class of applications we are placing. For example, if we restrict ourselves to consider only HPC applications or only J2EE applications in a cluster we can demonstrate that these properties hold. We ran two applications on the two testbeds for various mixes of background traffic picked from the HPC benchmarks. We observed the *Ordering Property* to hold for the two applications on the two testbeds. We noted (Fig. 4(b)) that the x3650 platform is more power-efficient (incrementally) for both the applications. This is true even when we changed the background traffic from *daxpy* to *fma* on the HS21 Blades. We also observe that the incremental power drawn by *daxpy* for both the background traffic at any given background load value is almost same. Hence, one can assume the *Background Independence Property* to hold at a coarse granularity in most